LOHEN: Layer-wise Optimizations for Neural Network Inferences over Encrypted Data with High Performance *or* Accuracy

Abstract

Fully Homomorphic Encryption (FHE) presents unique challenges in programming due to the contrast between traditional and FHE language paradigms. A key challenge is selecting ciphertext configurations (CCs) to achieve the desired level of security, performance, and accuracy simultaneously. Finding the design point satisfying the goal is often labor-intensive (probably impossible), for which reason previous works settle down to a reasonable CC that brings acceptable performance. When FHE is applied to neural networks (NNs), we have observed that the distinct layered architecture of NN models opens the door for a performance improvement by using *layer-wise* CCs, because a globally chosen CC may not be the best possible CC for every layer individually. This paper introduces *LOHEN*, a technique crafted to attain high performance of NN inference by enabling to use layer-wise CC efficiently. Empowered with a cryptographic gadget that allows switching between arbitrary CCs, LOHEN allocates layer-wise CCs for individual layers tailored to their structural properties, while minimizing the increased overhead incurred by CC switching with its capability to replace costly FHE operations. LOHEN can also be engineered to attain higher accuracy, yet deliver higher performance compared to state-of-the-art studies, by additionally adopting the multi-scheme techniques in a layer-wise manner. Moreover, the developers using LOHEN are given the capability of customizing the selection policy to adjust the desired levels of performance and accuracy, subject to their demands. Our evaluation shows that LOHEN improves the NN inference performance in both of these cases when compared to the state-of-the-art. When used to improve the CKKS-only inference, LOHEN improves the NN inference performance of various NNs 1.08–2.88×. LOHEN also improves the performance of mixed-scheme NN inference by 1.34–1.75× without accuracy loss. These two results along with other empirical analyses, advocate that LOHEN can widely help improve the performance of NN inference over FHE.

1 Introduction

Fully Homomorphic Encryption (FHE) [1] is one of the known privacy-preserving techniques that enable secure computation on encrypted data. To run an application using FHE, programmers first need to generate an FHE-based code that produces an output identical to that of the original (unencrypted) application code. Building secure and efficient FHE-based code poses unique challenges due largely to the differences between traditional programming paradigms and FHE's computation model [2]. One key challenge is selecting an appropriate *ciphertext configuration* (CC), a set of factors including vectorization and ciphertext parameters. The selected CC determines not only the security strength but also the overall performance and accuracy of the generated FHE code [3, 4], thus should be carefully chosen considering the structural properties of the original code. For example, if the original code contains SIMD (Single Instruction, Multiple Data) operations, a programmer tries to find an optimal packing option that aligns with the data structure and the inherent parallelism of the code in a way to maximize the benefits of SIMD computation [5].

The complexity of selecting CC escalates significantly as the code size increases. Large codes typically consist of multiple parts, each with its unique programming structures and properties. The presence of such mutually distinct structures and properties of parts within one large code makes it complicated or even impossible for one single CC to simultaneously be the best option for all single subsections. Thus, for practicality, conventional practice in prior work has been resorting to a solution, which involves choosing one reasonably good CC that provides acceptable (not necessarily the best) performance across the entire code. Consider a program that includes a subroutine F performing matrix-vector multiplication (MVM). With FHE in the ciphertext domain, F would be optimally computed with a CC whose batch size matches the dimensions of the MVM. However, if the program also has other subroutines dealing with lower (higher) dimensions of data, the conventional approach may adopt a CC targeted

to a smaller (larger) batch size for a greater cause [6]. This choice, while increasing computation latency for F alone, may enhance the overall performance of the program.

The drawback of using one CC is observed rather clearly when it comes to running neural network (NN) inferences over FHE. With the rapid growth of Machine Learning (ML) as a Service and increasing demands for privacy-preserving ML, researchers are actively exploring the use of FHE for NN models over encrypted data [3,4,7–10]. They often adopt the aforementioned approaches to select one single CC to be used globally across all the NN because such a CC still enables affordable performance for the entire NN inference process, while not the best option for some individual NN layer's computation within the model. Still, the unique characteristics of the NN inference computation, which is structured into clearly divided and concatenated parts (*i.e.*, layers), open the door to another approach as an alternative to traditional approaches in applying FHE to NN models. This approach chooses distinct locally-optimized CCs that are determined for each layer, taking each layer's data flow and operation types into consideration. Theoretically, this approach of executing each layer over encrypted data with its dedicated CC should promise better performance than when using one global CC across the entire NN. However, the practical application of this approach in NNs comes at a cost. It requires additional cryptographic algorithms to switch CCs between layers, incurring non-negligible computational overhead. As can be expected, in many cases, the CC switching overhead can outweigh the performance benefits, consequently making this layer-wise optimization approach more expensive than the traditional one that maintains a single CC throughout the entire NN inference computation.

This paper presents *LOHEN*, a technique that harnesses the performance benefits of layer-wise CCs and reduces the costs of CC switching, thereby making the layer-wise CCs more attractive than the globally chosen CC. To this end, we first introduce the notion of repacking operation which switches CCs, and design an efficient repacking operation for LOHEN called *LOHEN Cryptographic Repacking* (LCR). LCR is designed with an emphasis on efficiency and gives LOHEN more opportunity for performance improvement by having the capability of replacing other costly operations such as rotations and bootstrapping. LOHEN inserts LCR selectively only when the insertion is beneficial for performance, with the help of its cost analyzer. The cost analyzer is designed to estimate the performance impact of LCR insertions. Around this analyzer, LOHEN runs an algorithm that automatically determines where to insert LCRs and which CC to switch for better performance. Specifically, LOHEN first performs layer-wise optimization by determining locally optimal CKKS CCs for each layer and calculates the costs of CC switching between layers. Subsequently, it modifies or merges the CCs assigned to the layers in a way that minimizes the estimated total computation time, which is the same as the summation of the layer computation time and repacking time, for the entire NN inference.

LOHEN can also be directed for enhanced accuracy, by using multiple FHE schemes each having its own strengths and weaknesses in terms of accuracy and performance. For example, NN inference over CKKS must resort to the approximation when running non-arithmetic layers, inevitably inducing the accuracy loss [4, 8], because CKKS is not designed for non-arithmetic operations [11]. On the other hand, some schemes such as TFHE are ready for computing arbitrary functions including non-arithmetics, using a specialized routine called programmable bootstrapping (PBS) [12, 13]. Not enforcing the approximation, using such schemes enables the computation of non-arithmetic layers precisely. LOHEN is designed with the support for such a multi-scheme setup by extending the CC switching of LCR to enable cryptographic scheme conversion, incorporating a technique that is adapted from the one proposed earlier [14]. Taking the developer-specified accuracy goal, LOHEN selectively uses a different scheme for some non-arithmetic layers to reduce the accuracy degradation owing to the approximation. For example, a developer may ask LOHEN to prioritize the accuracy. For such developers, LOHEN uses the precision-oriented scheme for more layers to achieve the goal. If the developer seeks more performance enhancement, LOHEN generates an FHE program that achieves better performance in the trade of a small (but not bigger than the existing CKKS-only approaches) accuracy loss. It is noteworthy that the CC switching mechanism of LOHEN also brings significant performance improvements compared to the existing multi-scheme setups, even when running with the priority of accuracy.

To assess the effectiveness of LOHEN in performance enhancements, we implement LOHEN using a state-of-the-art FHE scheme, CKKS. As the precision-oriented scheme, our implementation of LOHEN uses TFHE. Our experiments using four different models (ResNet, VGG, SqueezeNet, MobileNet) and two datasets (CIFAR-10, ImageNet) demonstrate that LOHEN improves the performance of the state-of-the-art under all settings by enabling to use multiple CC. When used to improve the performance of NN inference over CKKS (*i.e.*, single scheme), LOHEN outperforms the existing ones 1.08–2.88×. LOHEN also improves performance in a multi-scheme setting. For example, LOHEN outperforms the state-of-the-art multi-scheme approach by 1.34–1.75× without accuracy reduction.

Our contributions can be highlighted as follows.

- We introduce the notion of repacking operation and design LCR, a repacking operation that provides CC-switching as well as additional benefits in replacing costly rotations and bootstrapping.
- We devise a cost analyzer and an algorithm that estimates the gains and losses associated with inserting LCR to determine where to insert LCRs and which CC to switch to.
- When applied to NN inferences using CKKS solely, LO-

Table 1: Notations

R_N	:	polynomial ring of degree N
s, ct	:	secret key & ciphertext encrypted with it
$\mathbf{m}, m(X)$	:	message vector & plaintext polynomial
LWE_q^n	:	LWE ciphertext (dimension n, modulus q)
$RLWE_{q,n}$	:	$RLWE$ ciphertext (degree n, modulus q)
λ	:	security level
h_i, w_i, h_o, w_o	:	height, weight of input/output tensor
c_i, c_o	:	number of channels input/output tensor
f_h, f_w, st	:	kernel size & stride of convolution

Table 2: Latency of CKKS operations (GPU, $N = 2^{17}, logq = 54$)

level (μs)	4	6	8	10	12	14
PMult	31	36	41	49	53	63
CMult	1004	1297	1624	2067	2462	2973
Rot(<<1)	927	1198	1504	1925	2293	2777

HEN outperforms existing state-of-the-art works up to 2.8× speedup while maintaining the same accuracy.

- When using multi-schemes, LOHEN enables zero loss in accuracy while still achieving 1.23–1.85× speedup compared to the CKKS-only works, demonstrating its benefits on both accuracy and performance.
- LOHEN allows developers to customize optimizations according to their specific needs, exploiting the trade-offs between performance and accuracy.

2 Background

This section describes the essential preliminaries of our work. We refer to the literature [11, 12] for cryptographic details. Table 1 lists the notations that we will use in this paper.

2.1 CKKS

CKKS [11] encodes and encrypts multiple fixed-point data elements into a single RLWE ciphertext, a process referred to as packing. Using the method, the arithmetics over RLWE operate in a SIMD-manner (*i.e.*, element-wise parallel computing). For a power-of-two integer $N \geq 2$, we define a polynomial ring with modulus q as $R_{q,N} = \mathbb{Z}_q[X]/(X^N+1) = R_N/qR_N$. The message vector $\mathbf{m} \in \mathbb{C}^{\frac{N}{2}}$ is encoded to a plaintext polynomial $m = \mathsf{Ecd}(\mathbf{m}) \in R_N$. The RLWE encryption of a polynomial m using the secret key $s \in R_N$ is computed as $(b,a) = (m+e-a\cdot s, a) \in R^2_{q,N}$, where $a \in R_{q,N}$ and $e \in R_N$ is chosen from a uniform distribution and distribution of a small standard deviation, respectively. $\langle ct, s\rangle$ denotes the RLWE decryption which is computed as $(b,a)\cdot(1,s) = m+e$. Operations over RLWE refer to operations between these polynomial forms – the bigger the used polynomial is (bigger N), the bigger overhead is induced. The representative CKKS operations are as follows :

Addition/Multiplication. The arithmetics between two ciphertexts ct_1 and ct_2 which encrypts the messages $\mathbf{m}_1$ and $\mathbf{m}_2$ results in a ciphertext that encrypts the element-wise arithmetic operation between $\mathbf{m}_1$ and $\mathbf{m}_2$. Table 2 depicts the latency of CKKS operations run on GPU. The latency of multiplication grows linearly with the level (lvl). The operations between ciphertexts (CAdd and CMult) are slower than those with plaintext (PAdd and PMult).

Rescale. After successive multiplications on a ciphertext, the scale is doubled (*i.e.* Δ^2). To avoid the accumulated scale from exceeding its upper bound Q, we need to perform the routine called rescale that reduces the current lvl.

Rotation (Rot). Rotation is a circular shift within the encrypted vector. Given an RLWE ciphertext ct that encrypts m, the rotation of ct by an offset k results in the encryption of m' where $\mathbf{m}' = \mathbf{m} << k$. Its cost increases along with the bit length of k, as shown in Table 3. This is because typical FHE libraries [15, 16] only generate rotation keys for offsets of power of 2, and use them sequentially to form other rotations.

Bootstrapping (BTS). When a ciphertext reaches a certain level that is unable to perform more multiplication, BTS should be performed to recover Q and lvl. Among many variants, we use the one that offers a modest precision [17], which requires a minimum of three lvl left in a ciphertext in order to initiate. BTS is generally known to take longer than a single multiplication at a scale of 2 to 3 orders of magnitude [18].

Table 3: Latency of rotations on (GPU, $N = 2^{17}, lvl = 7$)

offset (μs)	1	2	3	4	5	6	7
Rot	1360	1371	2396	1356	2382	2492	2431

2.2 FHE over the Torus (TFHE)

TFHE (also called CGGI) is a LWE/GSW-based FHE scheme. $LWE_q^n(\hat{m})$ denotes the LWE encryption of the message $\hat{m} \in \mathbb{Z}_q$ using the secret key $\hat{s} \in \mathbb{Z}_q^n$, computed as $(\hat{b}, \hat{a}) = (\hat{m} + \hat{e} - \hat{a}^\top \hat{s}, \hat{a}) \in \mathbb{Z}_q^{n+1}$ where $\hat{a} \in \mathbb{Z}_q^n$ and $\hat{e} \in \mathbb{Z}_q$ are randomly sampled. $\langle \hat{ct}, \hat{s}\rangle$ denotes a decryption, which is $(\hat{b}, \hat{a})\cdot(1, \hat{s}) = \hat{m} + \hat{e}$. The main feature that differs from CKKS is Programmable BTS (PBS) [12, 19], which allows to compute an arbitrary function, making TFHE adequate for non-arithmetic tasks [20–22]. Ironically, the critical downside of TFHE lies within PBS itself. As its name suggests, the operations are performed during PBS. For this reason, every operations performed over TFHE must be accompanied with PBS, which is essentially a kind of BTS [19]. Making things worse, TFHE cannot take advantage of the data-level parallelism because it does not support packing at the algorithm level.

2.3 Ciphertext Configuration (CC)

Figuratively, FHE ciphertexts are packaging boxes for the values they encrypt. Just as packaging boxes designed to fit for the products they hold in, FHE ciphertexts should be configured to fulfill the needs of the developers. Apart the scheme

Table 4: CC factors

$log\,q$	:	bitwidth of each modulus
q_0	:	base modulus
Q	:	ciphertext modulus at the highest level
Lv	:	max level of the ciphertext
N	:	degree of CKKS ciphertext

Table 5: The security requirements for $\lambda = 128$ [23]. For each N, Q should not be bigger than the presented number.

N	2^{10}	2^{11}	2^{12}	2^{13}	2^{14}	2^{15}
logQ	26	54	108	217	438	881

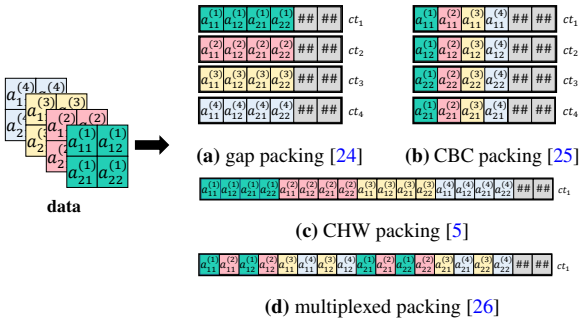


Fig. 1: Example of packing methods for data $(h_i, w_i, c_i) = (2, 2, 4)$

itself (CKKS/TFHE, or LWE/RLWE), FHE ciphertexts can be configured based on various CC factors, such as modulus chain and the degree of the polynomial, which determine not only the security level or data precision, but also the overall performance of the FHE code. The CC factors related to our paper are listed in Table 4. N and Q refer to the polynomial degree and coefficient bitwidth, respectively, which determine the security level λ following the (R)LWE problem. For each N, Q should not exceed a certain limit which is pre-defined by λ. Table 5 shows the case of $\lambda = 128$ [23].

The two factors also affect the performance. As FHE involves operations between polynomials, the bigger the coefficient Q and the polynomial degree N are, the higher the computational overhead gets. Usually, the time complexity of FHE operations is dominated by $O(N)$ or $O(N \log N)$ (the former related with polynomial addition, and the latter with multiplication). Additionally, N is also related to the maximum number of values that a RLWE ciphertext can pack, which affects performance in that the packed values in the encrypted vector of a RLWE ciphertext are computed in a SIMD-manner, as discussed in §2.1. Q consists of multiple small moduli, each being q_0 or q bits. Analogously, the ciphertext lose one q each time it performs rescaling. In other words, each ciphertext can compute as many operations as moduli it has, which refers to the maximum level (Lv). After each rescaling, the ciphertext loses a lvl.

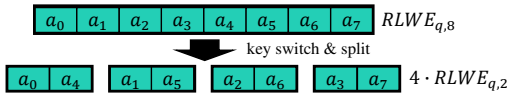


Fig. 2: Example of Ringswitch to lower degree

Table 6: Latency of Ringswitch to/from n/N.

n/N (μs)	2^{14}	2^{15}	2^{16}
2^{14}	-	89	161
2^{15}	89	-	161
2^{16}	161	161	-

2.4 CC Selection

The custom in FHE computing is to select one CC to be used as a global (uniform) configuration during the whole FHE computation. This is because, to the best of our knowledge, no prior work has suggested a method to change CC during FHE computation in a cryptographic way, and the CC could only be determined when the input is encrypted. Researchers have been suggesting methods on how to select and use a CC throughout the computation (global CC) for efficient computation of a given workload [2, 5]. However, it is difficult (probably impossible) to determine which CC suits the best for a given general computation, as the factors and their relevant effects are interconnected in a complicated way. For example, using a small N and Q gives better cost for individual FHE operations, but would require frequent BTS. Also, whereas vectorization (*i.e.*, data alignment) is important to maximize the benefits of SIMD, re-alignment of data during computation requires lots of rotations, whose costs are also non-negligible. For such reason, existing works [6, 18, 27–29] chose to prioritize certain factors over others, mostly focusing on the vectorization or scheduling specific FHE routines.

Several works introduced manual global CC selection approaches retrofitted to specific applications of their interests. The representative application is NN, which was the target of numerous works [3, 8, 30, 31]. Each of the works introduces the CCs it used, including the vectorization (packing) methodologies depicted in Fig. 1. Among them, FHE-MP-CNN [8] is the state-of-the-art NN inference work using CKKS that shows the best balance between performance and accuracy by selecting a CC with the goal of maximizing the slot utilization (*i.e.* minimizing the number of ciphertexts), which we use as the baseline CC selection used throughout our design and evaluation of LOHEN. Hyphen [30] also introduces a method, which we will hereafter call *LoLat*, tailored to minimize the rotation and bootstrapping costs in trade for accuracy.

2.5 LWE/RLWE Conversions

LWE/RLWE conversions have been widely dealt with by various works [14, 32, 33]. In fact, TFHE [12] itself uses both LWE and RLWE during its computation, which motivated the

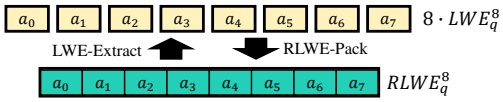


Fig. 3: Example of LWE-Extract and RLWE-Pack.

other works to devise enhanced algorithms for conversions between LWE and RLWE. Below, we introduce the common building blocks used in their works. Note that previous works focused on enhancing the scheme conversion itself. We describe how we exploit these building blocks to perform CC switching and introduce other functionalities LOHEN offers for adaptive performance enhancement of FHE computation.

Ringswitch. Ringswitch [34, 35] is an operation that performs transitions between rings with different degrees N/n (large/small rings). The conversion can be processed in both directions by splitting one ciphertext into multiple small ones or the other way around. Fig. 2 depicts an example of a ring with degree 8 is switched to a ring with degree 2. Our work follows Ringswitch explained in literature [34], of which the latencies are depicted in Table 6 – note that the latency is related to the bigger N, whether it is the destination or the source. One important issue is that increasing N enables the usage of bigger Q leading to bigger Lv, but Ringswitch does not naturally increase Lv. To do so, another operation is required to recover the ciphertext back to its maximum capacity, which has similar functionality as BTS as well as latency.

LWE-Extract. With respect to the coefficient-encoded RLWE ciphertext in $R_{q,n}^2$, one can extract the values of individual coefficients in its plaintext as n LWE ciphertexts in $\mathbb{Z}_q^{n+1}$. The function $\mathsf{Extract}^i$ [12] extracts the i-th coefficient m_i of the plaintext polynomial $m(X) \in R_{q,n}$, such that $m(X) = m_0 + m_1 X + \cdots + m_{n-1} X^{n-1}$ holds.

RLWE-Pack. This operation [12] packs n LWE ciphertexts in dimension n into an RLWE ciphertext. This assigns the message m_i of the i-th LWE ciphertext to the i-th coefficient of plaintext $m(X)$ of a single RLWE ciphertext. It is recognized as an inverse routine of LWE-Extract, as depicted in Fig. 3.

2.6 NN Notations

Throughout our literature, we denote the inference tasks following the rules described below. The first letter represents the dataset (C, I for CIFAR-10, and ImageNet, respectively), and the second letter states the NN architecture (R, V, SQ, and MO for ResNet, and VGG, SqueezeNet, and MobileNetv2). The number appearing right after these two letters is the depth of the NN architecture if necessary. For example, CR20 stands for ResNet-20 using the CIFAR-10 dataset, and ISQ stands for SqueezeNet using the ImageNet dataset.

Table 7: FHE-MP-CNN evaluations using GPU.

seconds(%)	Rotation	Others	BTS	SUM
CR20	4.08 (20%)	4.69 (24%)	11.32 (56%)	20.10
IR18	266.40 (68%)	24.09 (6%)	101.91 (26%)	392.40

Table 8: Structural factors $w_o \times h_o, c_o$ in CR20 and IR18

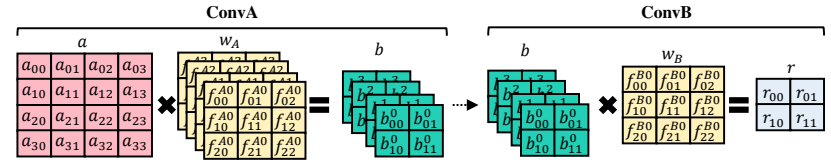
Layer Group	Conv-1	Conv-2	Conv-3	Conv-4	Conv-5
CR20	32× 32, 16	16× 16, 32	16× 16, 32	8× 8, 64	8× 8, 64
IR18	112× 112, 64	56× 56, 64	28× 28, 128	14× 14, 256	7× 7, 512

3 Motivation

This section describes the three motivations behind our design of LOHEN, and compares it with the prior works about using multiple FHE schemes. As the first motivation, we describe the case where layer-wise CCs are potentially more beneficial than the global CC. Subsequently, we show that the cost of switching CC between layers could be lower than expected due to the computational cost of rotation operations, which LOHEN's CC switching operation can replace. As the third, we present a case showing that replacing all rotations with CC switching degrades the performance. We also compare LOHEN with prior works [14, 33] that proposed and used a mechanism to convert a CKKS ciphertext into a TFHE one and vice versa.

Potential Benefit of Layer-wise CC. Using layer-wise CCs that are chosen for individual layers is more beneficial to reducing the inference latency than using one global CC for all layers, as discussed in §2.2. Fig. 4 illustrates an example of two consecutive convolutions executed over CKKS using the multiplexed parallel computation method [8], which is the state-of-the-art in efficiently executing convolution over CKKS. Fig. 4a visualizes the two convolution layers executed on the plaintext domain, and Fig. 4c and Fig. 4d show how the NN with the two layers is executed over CKKS when we choose N to be 16 and 8, respectively. Fig. 4b shows the relative computational cost, which is proportional to the execution latency (lower is better) of each convolution layer under two different CCs, computed using the normalized values from Table 2. Comparing the total cost, 16 is a better choice if we are constrained to use the same value for N. On the contrary, using 8 for ConvB and 16 for ConvA brings better performance when we have the option to choose the value of N for each layer differently without additional cost. This observation suggests that using layer-wise CC could improve the performance of NN inference over CKKS if we can enable such CC switching efficiently enough.

Possibility of Efficient CC Switching. The ability of LOHEN's CC switching mechanism (see §4.2) to replace the rotation operation enables LOHEN to switch CC efficiently enough to make the layer-wise CC approach truly beneficial for performance. NN inference is done by executing a series of layers where the output vector from one or more layers constitutes the input vector of another. In most cases, the output

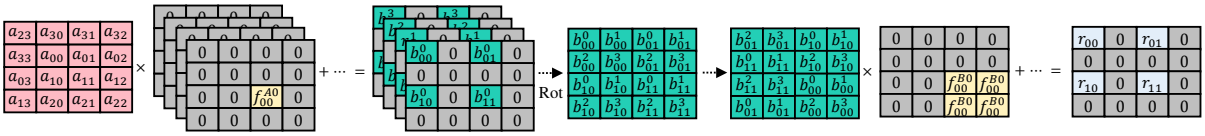


(a) ConvA with $(c_o, f, st, p) = (4, 3, 2, 1)$ and ConvB with $(c_o, f, st, p) = (1, 3, 1, 1)$

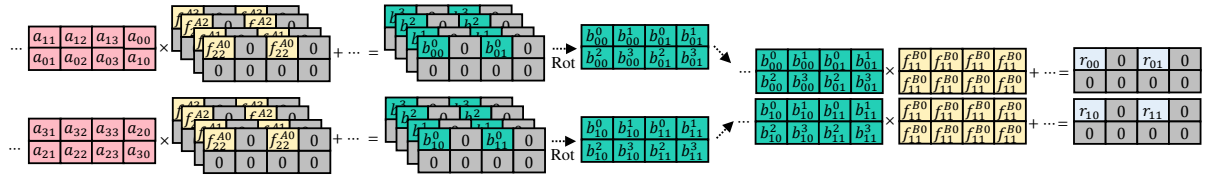
# of OP.	N=16			N=8		
	Mult	Rot	Rel. Cost	Mult	Rot	Rel. Cost
ConvA	9	6	378	72	16	552
ConvB	9	11	678	18	19	588
Total			1056			1141

(b) Cost Comparison. 'Rel. Cost' refers to relative cost, which is proportional to the execution latency, *i.e.*, lower is better

.



(c) global CC with N=16



(d) global CC with N=8

Fig. 4: Example of consecutive convolution layers computed over different N. We assume other factors (*e.g.*, Q) to be even.

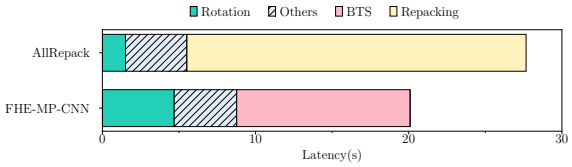


Fig. 5: Latency of CR20 using Layer-wise CC and FHE-MP-CNN

vector cannot be used as the input vector as it is. It must be reconstructed, *i.e.*, manipulated with a series of permutations to take full advantage of SIMD parallelism. Over FHE, this is done by a series of rotation operations, whose cost increases with the length of vectors. Fortunately, such a reconstruction of vectors can be done in the process of CC switching (detailed in §4.2), thereby effectively reducing the latency cost of CC switching depending on the cost of rotations it is replacing. Our empirical study suggests that the cost reduction could significantly contribute to the performance benefit of CC switching. As Table 7 shows, a large portion of encrypted NN inference latency is spent for rotation operations, leaving non-trivial chances for LOHEN's CC switching to replace them also to enable the use of layer-wise CC. Some structural factors presented in Table 8 suggest that the cost of rotation increases with the vector dimension, *i.e.*, there are more chances to use CC switching when running NNs for wider vectors.

The Necessity of Selective CC Switching. Replacing rotations with CC switching operations does not always affect the NN inference performance positively. Fig. 5 shows the latency breakdown of CR20 inference before and after replacing the rotations between all layers with the LOHEN's CC switching operation (noted as "AllRepack"). Even with

Table 9: Comparison between LOHEN and works on ciphertext conversions.

Feature	PEGASUS [14]	HEIR [33]	LOHEN (LCR)
FHE scheme conversion	✓	✓	✓
Switching CCs of the same scheme	✗	✗	✓
Replacing Rotations with Repacking	✗	✗	✓
CC selection	Global	Global	Layer-wise
mixed FHE scheme	Enabled	Utilized	***A**

*A: Latency improved, enable to trade accuracy with latency

the help of locally fitted CC and reduced overhead for rotations, the end-to-end latency becomes longer than the one with global CC, suggesting that CC switching improves the inference latency only when we insert the CC switching operations selectively. LOHEN addresses this by using its cost analyzer and an algorithm to automatically determine which rotations it replaces with its CC switching operations.

Comparison with Prior Works. Table 9 shows similarities and differences between LOHEN from the two prior works [14, 33] about ciphertext conversions. We present this comparison because LOHEN not only switches CCs but can also convert ciphertext schemes by extending LCR with some mechanisms from them after adaptation. As the table shows, the two prior works are focused on converting the FHE scheme, in an attempt to precisely execute the non-arithmetic layers. They only use one global CC for each of the schemes, leaving unexplored the possibility of using layer-wise CCs, which is the primary focus of LOHEN. Still, LOHEN moves

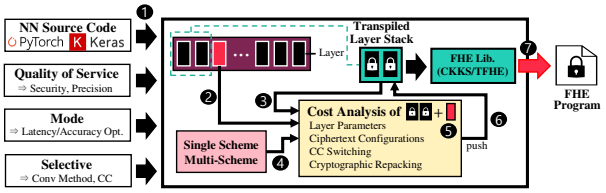


Fig. 6: LOHEN Overview

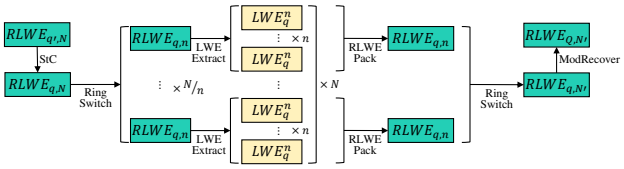


Fig. 7: Illustration of LCR

Table 10: BTS and LCR latency on GPU (μs).

logN	15	16	17
BTS	33,766	96,964	371,541
LCR	47,777	111,012	402,749

beyond the layer-wise CC and considers the existing ciphertext conversion techniques as a useful building block. Specifically, LOHEN uses an adapted version of ciphertext conversion techniques designed by the two prior works together with its LCR to enjoy both improved accuracy and reduced inference latency (§4.5).

4 LOHEN Design

The observations introduced in §3 have driven us to design LOHEN which selectively performs CC switching and FHE scheme conversion to reduce the NN inference latency and bound the accuracy loss. To this end, we designed a new cryptographic gadget that switches CC and can replace rotation, called LCR (§4.2). LOHEN uses this LCR as a new building block to improve the inference performance automatically by selectively inserting it and replacing rotations with it (§4.3), with the help of its cost analyzer (§4.4). Furthermore, LOHEN also uses the FHE scheme converters (§4.5) as another building block and selectively inserts them if it is instructed by the developer to achieve a certain accuracy level (§4.6).

4.1 Overview

Fig. 6 gives an overview of LOHEN. LOHEN takes a description of NN computation (*i.e.*, the source code) written in machine learning frameworks such as PyTorch or Keras as the input. The source code is first transformed into an intermediate format that summarizes each layer's characteristics, such as kinds of operations or convolution filter types (❶). From this layer-wise information, LOHEN uses its analyzer to estimate the cost of each building block and uses the result to select and schedule the FHE operations for each layer, one by one, from the first layer. To analyze, estimate, select, and schedule the FHE operations for the next layer, LOHEN takes the intermediate result that contains the results for the earlier layers (❷) and the composition of the layer (❸) as the input and analyzes the cost (❺). Through this analysis, LOHEN determines the choice of the CC that brings better performance and pushes to the intermediate results (❻) for the following steps. The candidates of the operations considered in these steps include LCR (§4.2) for switching CC and TFHE-based non-arithmetic layers (§4.5) for mitigating the accuracy loss (❹). Once all layers are analyzed, LOHEN emits a FHE program corresponding to the computation (❼).

During these steps, the cost analyzer automatically considers various constraints such as the target security level (*i.e.*, λ), precision (*e.g.*, 32-bit), or the user's desired optimization priority (*i.e.*, latency or accuracy). §4.4 describes more details about the cost analyzer. §4.3 and §4.6 explains the algorithm.

4.2 LOHEN Cryptographic Repacking (LCR)

Fig. 7 illustrates LCR, designed for LOHEN to enable CC switching and to replace the rotation-based permutations between layers. Taking an RLWE ciphertext as an input, the repacking gadget performs SlottoCoeff (StC) [36], which changes a slot-represented ciphertext into coefficient form. Followed by this StC, LOHEN should run the LWE-Extract and RLWE-Pack for LCR. A key design decision to enhance performance involves the use of Ringswitch immediately after StC because the subsequent operations exhibit reduced latency when applied to ciphertexts of lower degrees. Specifically, LCR gadget splits a RLWE ciphertext with degree N into multiple RLWE ciphertexts with the degree $n = 2^{11}$, which is almost always lower than the degree N of the input ciphertext. We choose 2^{11} as the target degree because it is the smallest degree that provides non-zero multiplicative depth under the 128-bit security (*i.e.*, $\lambda = 128$) when $q = 54$, both of which are the de-facto standard choice of parameters. Note that the optimal target degree may vary when these two changes. LOHEN runs LWE-Extract and RLWE-Pack in sequence to extract the LWE from RLWE ciphertexts, and then pack the latter into one or more RLWE ciphertexts back. The last step of LCR is the modRecover that recovers lvl of the target ciphertexts, which has the same functionality as BTS.

CC Switching by LCR. LOHEN capitalizes on three fundamental aspects of LCR strategy. First, the aforementioned design choice enables LOHEN to switch the ciphertext degree efficiently when it performs LCR. LCR already performs Ringswitch while reconstructing the RLWE ciphertext, and LOHEN can choose to use a degree that is not the same as that of the original ciphertext if its cost analyzer reports that using a new degree delivers lower latency. Second, LCR also obviates the rotations between layers because weaving

Algorithm 1: Layer-wise cost estimation analysis.

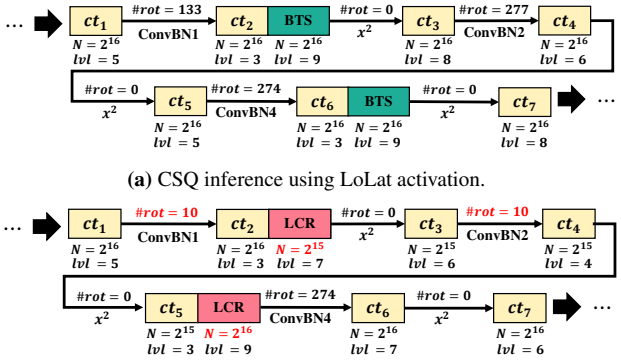
Input: *CLS*: Closed Layer Stack, *ALS*: Active Layer Stack, l_k : current layer
Output: *ALS*, *CLS*, l_{next}

1 **Init**
2 $Cost_{static}(l_k, N) = A_k \cdot BC_A \cdot k_A(N) + P_k \cdot BC_P \cdot k_P(N)$
3 $+C_k \cdot BC_C \cdot k_C(N) + R_k \cdot BC_R \cdot k_R(N)$
4 $Cost(l_k) = (Cost_{static}(l_k) + Cost_{dyn}(l_k))$
5 **if** *ALS.getRemainlvlSlot*(l_k) == *Minlvl(ALS,* l_k) **then**
  // need to repack or enlarge N
6 **if** *EnlargeCost*(l_k) < *RepackCost*(l_k) **then**
7 ALS.expand($N, N+1$); ALS.push(l_k);
8 $Cost(l_k) \times = k(N+1)/k(N)$;
9 ALS.cost × = (1+1/ALS.lvl);
10 ALS.cost += $Cost(l_k)$; $l_{next} = l_{k+1}$;
11 **else**
12 $l_k \rightarrow PERM = 0$; ALS.cost += RepackCost(l_k)
13 ALS.push(Repack); $l_{next} = l_k$
14 CLS.push(ALS); ALS ← new ALS;
15 **else**
  // push L to ALS without repack
16 **if** $Cost_{dyn}(l_k)$ + *ALS.cost* × *(1/ALS.lvl)* < *RepackCost*(l_k) **then**
17 ALS.cost × = (1+1/ALS.lvl);
18 ALS.cost += $Cost(l_k)$; ALS.push(l_k); $l_{next} = l_{k+1}$;
  // push repack to ALS and close the stack
19 **else**
20 $Cost_{dyn}(l_k) = 0$; ALS.cost += RepackCost(l_k);
21 ALS.push(Repack); $l_{next} = l_k$
22 CLS.push(ALS); ALS ← new ALS;

LWE-Extract and RLWE-Pack enables us to perform arbitrary permutations instead of rotations. Third, LCR also involves the role of BTS because it recovers *lvl*. LCR alone is not an efficient replacement for BTS, as Table 10 shows. However, this aspect potentially makes the cost of adopting LCR lower, along with the second aspect, when it replaces both BTS and CC switching.

4.3 Layer-wise CC and Operation Selection

As explained earlier, LOHEN determines the CC and the corresponding operations for each layer using the results from the earlier ones. In this context, Algo. 1 shows how LOHEN works for the k-th layer, denoted as l_k in the algorithm. Inputs to this algorithm include the Closed Layer Stack (CLS) and Active Layer Stack (ALS). CLS is a stack of layers for which LOHEN already determined the CC, and ALS contains the ones that LOHEN may still change their CC. After executing this step, LOHEN may push l_k to the ALS and move on to the next layer (l_{k+1}) or decide to perform LCR and assign a CC for l_k which is different from ALS. If the latter happens, it pushes all elements from ALS to the CLS and empties the ALS. In brief, Algo. 1 chooses the better option from the two (local optimal choice) for each layer iteratively. Both ALS and CLS are empty when LOHEN handles the first layer, and CLS contains all layers after LOHEN finishes handling the last layer.



(a) CSQ inference using LoLat activation.

(b) CSQ inference with LOHEN

Fig. 8: Example of our cost analysis during CSQ Inference.

The algorithm begins by computing the basis latency cost of a newly incoming layer, l_k, denoted as $Cost(l_k)$ (line 4). It subsequently compares the required multiplicative depth, or *lvl*, when this l_k is pushed to ALS and the remaining *lvl* considering the currently chosen candidate CC for ALS (line 5). If ALS will have sufficient budget for multiplication (*i.e.*, *lvl*) left even after having l_k, the algorithm moves to compare the cost of LCR and rotation-based permutation (line 16). Specifically, LOHEN pushes l_k to ALS and moves on to the next layer if LCR is more expensive. If LCR is found to be more efficient, it pushes LCR to ALS and revisits l_k. After pushing LCR to ALS, LOHEN closes ALS and appends it to CLS so that it can start from a fresh and empty ALS because the added LCR operation will fresh out the CC and recover *lvl*.

If the addition of l_k to ALS consumes all remaining *lvl*, *i.e.*, requires the execution of BTS right after l_k, LOHEN considers the increase of degree (N) and LCR as two possible options. Specifically, LOHEN compares the additional latency cost of ALS if the degree is enlarged for the entire ALS, including l_k, with the cost of LCR (line 6). If LCR is found to be more efficient, LOHEN pushes LCR to ALS, pushes it to CLS, and starts again for l_k with fresh and empty CLS (line 12–line 14). Otherwise, it changes the CC of ALS, *i.e.*, enlarges the degree, and pushes l_k to ALS. Note that this can be done because enlarging N increases the *lvl* budget of the ALS. Fig. 8 depicts an example of how our cost analysis works for the CSQ workload. Using the LCR, LOHEN reduces the number of rotations in ConvBN1 and ConvBN2 to 10 from 133 and 277, respectively. It also reduces the ring size of ct_3, ct_4, and ct_5 to $N = 2^{15}$.

Claim (Effectiveness of Algo. 1)**.** *Algo. 1 guarantees to determine the CC for each layer and the locations of LCR such that the resulting FHE program outperforms the one using global CC (i.e., using the same CC for all layers).*

Proof. For each layer, the algorithm makes a choice out of two options. For the k-th layer l_k, LOHEN can either group l_k with the previous layers in ALS to use the same CC (**C1**) or perform LCR and compute l_k using a different CC than the former layers (**C2**), where each of the two choices involves additional costs (denoted as $C1_k$ and $C2_k$). Note that choosing **C1** for all layers gives the same result as global CC, while LOHEN may also choose **C2** for several layers.

Let G_k and L_k be the latency of computing the first k layers using global CC and CCs from LOHEN, respectively. Then we have $G_0 = L_0$ as their cost is 0. If $G_k \geq L_k$, then $G_{k+1} \geq L_{k+1}$ also holds because $G_{k+1} = G_k + C1'_{k+1} \geq L_k + C1'_{k+1} \geq L_k + C1_{k+1} \geq L_{k+1}$. The first equality is from G_{k+1} being the result of choosing **C1** after G_k, with additional latency $C1'_k$. The second inequality holds because $G_k \geq L_k$ from our assumption. The third inequality ($L_k + C1'_{k+1} \geq L_k + C1_{k+1}$) holds if $C1'_{k+1} \geq C1_{k+1}$. This is true because $C1'_{k+1}$ is the additional latency when l_{k+1} adopts the N of existing ALS, and this N may not be the best one for the l_{k+1}, but $C1_{k+1}$ is the latency when l_{k+1} chooses the best possible CC, as the first layer of newly beginning ALS. The last inequality ($L_k + C1_{k+1} \geq L_{k+1}$) holds because L_{k+1} is the sum of L_k and the better cost option from $C1_{k+1}$ and $C2_{k+1}$. By induction, $G_n \geq L_n$ holds for any integer $n \geq 0$ because $G_0 = L_0$ and $G_{k+1} \geq L_{k+1}$ if $G_k \geq L_k$ □

4.4 Estimating Layer- and ALS-wise Cost

The cost analyzer is a key building block that LOHEN relies on. It estimates the cost in terms of inference latency for either one layer (*e.g.*, l_k) or a stack of layers (*e.g.*, ALS). To estimate the cost of a layer l_k, we use the number of primitive operations composing this layer to obtain the basis cost and multiply it with the degree-dependent factor. We first estimate the basis cost by combining the static and dynamic portion of the cost. The static portion comes from Add, PMult, CMult, and some Rotation operations because these are executed regardless of how the layer is combined with the others. The dynamic portion comes from the cost of permutation, which is also composed of rotations. We consider the cost of permutation as dynamic because its cost may vary depending on the way this layer is composed – their numbers are determined by the approach we adopt (FHE-MP-CNN in our default settings). LOHEN estimates the static cost using the number of static operations and their individual costs, which are empirically profiled. It estimates the dynamic cost ($Cost_{dyn}(l_k)$) using the number of rotations required and BC_R, which is reset to 0 if the layer is to be preceded by the LCR. (line 20). Accordingly, LOHEN estimates the cost for each N using the asymptotic time complexity of each operation.

$$Cost_{static}(l_k, N) = A_k \cdot BC_A \cdot k_A(N) + P_k \cdot BC_P \cdot k_P(N) + C_k \cdot BC_C \cdot k_C(N) + R_k \cdot BC_R \cdot k_R(N)$$

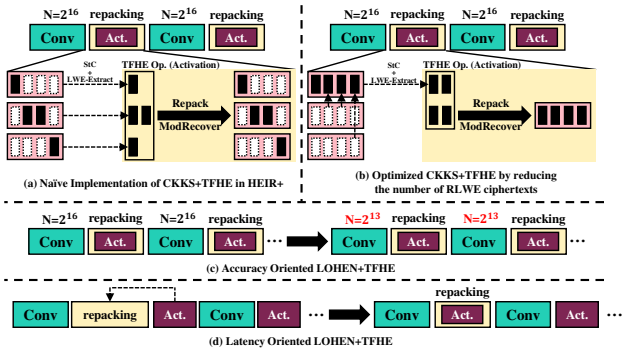


Fig. 9: Layer-wise Multi-Scheme Approaches

A_k, P_k, C_k, and R_k are the number of each operation in l_k and BC_A, BC_P, BC_C, and BC_R are their costs. N is the degree of the ciphertext, and k_A, k_P, k_C, k_R are the degree factors that have the following form, considering the time complexity.

$$k_A(N) = k_P(N) = \frac{N}{2^{11}} \quad k_C(N) = k_R(N) = \frac{N \log N}{2^{11} \cdot 11}$$

4.5 LOHEN Scheme Conversion Gadget

LOHEN is also equipped with a cryptographic gadget that enables it to mitigate the accuracy loss by executing non-arithmetic layers over TFHE. This gadget is an adaptation of LCR using the techniques proposed in prior works [14,32,33] with the ability to convert LWE ciphertexts to RLWE ones and vice versa. Fig. 9a and Fig. 9b show how we adapt the existing scheme converting gadgets. Unlike the ones used by the prior works, LOHEN's gadget combines multiple sparse gadgets into a dense one before the actual scheme conversion so that the result of TFHE layers can be converted into a single RLWE ciphertext. This change reduces the number of ModRecover operations, whose latency takes a large portion of the gadget's latency. In the example shown in Fig. 9a and Fig. 9b, the number of ModRecover is reduced from 3 to 1. The design also helps gather the scattered values in one ciphertext before engaging in LCR, reducing the amount of LWE-Extract and other components. Note that we *do not* consider this as a primary contribution of LOHEN but rather a way to enhance the multi-scheme baseline's performance we use in our evaluation in their favor, *i.e.*, we compare LOHEN with the prior works that are equipped with the same performance improvement technique.

4.6 Selective TFHE Application

Utilizing the additional TFHE-based non-arithmetic layers expands the design space, which requires again an automated approach to determine where to use the TFHE-based layers and how to set each layer's CC. The TFHE layers bring improved accuracy but are slower than their counterpart using

CKKS [14, 37]. Replacing approximated non-arithmetic layers with TFHE-based ones is, therefore, an act of paying the latency for accuracy, but it also introduces chances to switch CC. LOHEN explores this additional design space by using both the TFHE-based layers and the LCR when producing FHE programs. The developers can determine how much LOHEN should use TFHE layers, fulfilling their own desire for performance and accuracy. We showcase three examples on how developers can direct LOHEN according to their own choice of trade-off between performance and accuracy.

Goal 1: Accuracy-oriented. The first possible direction by the developer is to prioritize accuracy over performance, *i.e.*, to be as precise as possible. LOHEN achieves this all by inserting TFHE layers into all non-arithmetic layers. This ensures the best possible precision because all the non-arithmetic layers are computed with exactness over TFHE. After allocating TFHE computation for the non-arithmetic layers, LOHEN performs its layer-wise CC selection for the remaining ones. Fig. 9c depicts an example running LOHEN in the accuracy-oriented mode, where the two activation layers are replaced with the TFHE layers. This enables LOHEN to switch CC at the same points because LOHEN's conversion gadgets also perform the repacking as LCR does. LOHEN uses Algo. 1 to each of the separated groups of arithmetic layers (single convolutions in the example) to choose layer-wise CC. As a result, each convolutions is set with $N = 2^{13}$, the smallest N having $lvl = 4$, since we need $lvl = 3$ to initiate the subsequent TFHE layers with repacking.

Goal 2: Latency-oriented. The second option that is available for the developers is to prioritize latency, with an unbounded budget for accuracy loss. Under this constraint, LOHEN inserts the TFHE layers only if its latency impact is expected to be small. Specifically, it inserts repacking as it would be without considering the TFHE layers and considers the non-arithmetic layers adjacent to the inserted repacking as opportunities for accuracy improvement. Such non-arithmetic layers can be replaced with TFHE-based ones with small performance degradation because they can be merged with the adjacent repacking gadget that LOHEN has already inserted, *i.e.*, inducing no other extra cost for LWE-Extract, RLWE-Pack or ModRecover. This approach may improve the model accuracy when compared to the result of LOHEN without TFHE layers, but the resulting FHE program's latency becomes longer because the non-arithmetic layers over TFHE are slower than those over CKKS. Fig. 9d illustrates an exemplary result of this approach, assuming that LOHEN placed a repacking after the first convolution layer. Note that the developers who seek even better inference latency could also instruct LOHEN not to use TFHE layers at all.

Goal 3: Balanced. The third option, which is also further tunable, is the balanced priority where the developer can fine-tune the amount of accuracy loss to be paid for the performance gain. Behind this option is the observation that inserting more TFHE layers increases the latency but is likely to improve accuracy. Using the cost estimations presented in the previous section (§4.4), developers can determine the performance loss or gains from their decisions in trade of higher or lower accuracy. To assist developers in their decisions, LOHEN provides a profiler that assesses the model accuracy when replacing the first k non-arithmetic layers by changing k. It speculates the model's accuracy for each k by measuring the accuracy with 1000 queries; its results when LOHEN runs for ISQ and IMO are shown in Fig. 10. This result presents the smallest value of k that gives speculated accuracy the same as the encrypted model (*i.e.*, no accuracy loss even if there are some non-arithmetic layers computed over CKKS). Fundamentally, this balanced goal has the best-effort nature and may result in replacing all non-arithmetic layers with the TFHE ones. Nevertheless, we could obtain non-trivial results for the two workloads, which are 5 and 10, as Table 11 shows. From the value, developers can reduce the number of layers to be computed over TFHE, which leads to higher performance for a bit of accuracy loss (*e.g.*, computing IMO with 5 TFHE layers achieves 1.13× speedup for 1.2% accuracy loss). We use these results for the evaluation later. Note again that the result of this balanced goal is not guaranteed to be different from the other goals, and we decided to replace the non-arithmetic layers appearing earlier, referring to prior work on the relationship between the noisy layers and model accuracy [38].

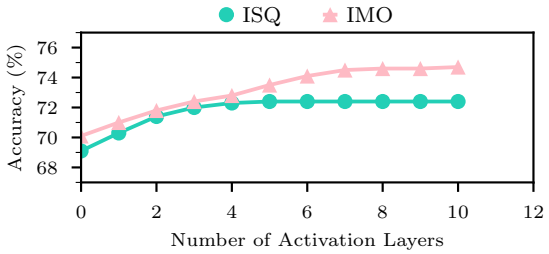


Fig. 10: Impact of the number of activation layers run over TFHE to accuracy over ImageNet inferences.

Table 11: Latency of LOHEN according to the number of activation layers computed with TFHE.

	# layers in TFHE	0	5	10	…	ALL	Baseline (w/o. LOHEN)
ISQ	Latency(s)	1008	**1297**	1502	…	1690	2093
	Acc. Loss	-3.3	**0**	0		0	-3.3
IMO	Latency(s)	1016	1392	**1573**	…	2056	2923
	Acc. Loss	-4.6	-1.2	**0**	0	0	-4.6

5 Evaluation

We answer the following about the effectiveness of LOHEN.

- Does LOHEN improve the performance when applied to the state-of-the-art CNN over CKKS? (§5.1, §5.3)

Table 12: The NNs used for our evaluation (Notations from §2.6).

Name	Dataset	NN Architecture	Reference
CR20	CIFAR-10	ResNet-20	[39]
CV16	CIFAR-10	VGG-16	[40]
CSQ	CIFAR-10	SqueezeNet	[41,42]
IR18	ImageNet	ResNet-18	[39]
ISQ	ImageNet	SqueezeNet	[42]
IMO	ImageNet	MobileNetv2	[43]

- Does LOHEN also improve the performance of the latency-optimized variant? (§5.2, §5.3)
- Can LOHEN incorporate a TFHE-based non-arithmetic layer for balancing the latency and NN accuracy? (§5.4)

Datasets and NNs. As the workloads for evaluation, we use the combinations of four widely used NNs (ResNet, VGG, SqueezeNet, and MobileNetv2) and two widely used datasets (CIFAR-10 and ImageNet). Table 12 presents the details of the combinations and their names that we use.

Accuracy Measurement. Unless otherwise specified, we use 1,000 queries from the validation set to obtain all accuracy numbers we report in this paper, and the accuracy refers to Top-1 accuracy. Note that this is a widely adopted method to measure the CNN inference accuracy.

Experimental Environment. We run all experiments using a machine with two Intel Xeon Gold 6326 CPUs, 1TB of main memory, and one NVIDIA RTX 3090 GPU with 24GB of memory. We also used unified virtual memory (UVM). As the underlying FHE library, we used a customized version of the GPU library called Liberate-FHE [44]. We chose Liberate-FHE instead of SEAL, which FHE-MP-CNN [8] used for several reasons. First, SEAL does not natively support BTS. The SEAL variant that the authors of FHE-MP-CNN [8, 45] implemented for their work exhibits sluggish BTS compared to other implementations [15]. Also, Liberate-FHE is fully programmable for GPU. We make sure to compare ours with FHE-MP-CNN fairly by running it using our version of Liberate-FHE and comparing LOHEN against this result–the end-to-end latency of FHE-MP-CNN is significantly reduced, making it a more appropriate baseline for our study. We also customized Liberate-FHE to support the FHE operations introduced in §2 in order to support full functionalities of CKKS efficiently using GPU. We modify this to enable Ringswitch, which changes the ring size, following the way introduced in a prior work [34]. We bind Liberate-FHE with NuFHE [46], a TFHE library for GPU.

5.1 Effectiveness on FHE-MP-CNN

FHE-MP-CNN. We evaluate the effectiveness of LOHEN when it is applied to FHE-MP-CNN [8] because it is considered as the state-of-the-art in encrypted CNN inference over CKKS, balancing the accuracy and latency by using its unique multiplexed packing and high-precision ReLU approximation, enabling to use pre-trained models [47].

Table 13: End-to-end latency of LOHEN applied to FHE-MP-CNN. AllRepack replaces all rotation-based permutations with LCR

Workload	FHE-MP-CNN	AllRepack	LOHEN
CR20	20.10s	29.35s (1.46×)	18.68s (0.93×)
CV16	72.60s	82.04s (1.13×)	46.65s (0.64×)
CSQ	44.42s	51.53s (1.16×)	32.75s (0.74×)
IR18	392.40s	302.15s (0.77×)	194.60s (0.50×)
ISQ	2092.62s	1757.80s (0.84×)	1007.96s (0.48×)
IMO	2922.78s	2279.77s (0.78×)	1016.17s (0.35×)

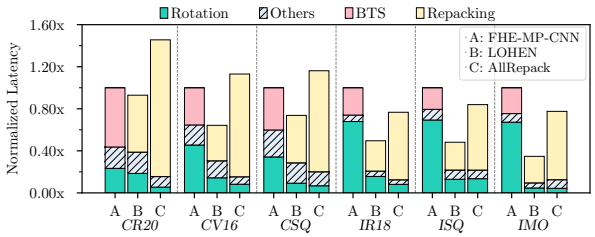


Fig. 11: Latency Breakdown of LOHEN(B) and AllRepack(C) applied to FHE-MP-CNN(A) (Normalized to FHE-MP-CNN).

Table 14: The accuracy of workloads when running over plaintext space with different activation functions.

Workload	ReLU	x^2 (Used For LoLat)
CR20	91.8%	90.1% (-1.7%, 0.98×)
CV16	94.4%	92.8% (-1.6%, 0.98×)
CSQ	89.4%	88.1% (-1.3%, 0.99×)
IR18	70.2%	65.6% (-4.6%, 0.93×)
ISQ	72.4%	66.2% (-6.2%, 0.91×)
IMO	74.7%	67.4% (-7.3%, 0.90×)

End-to-end Inference Latency. Table 13 shows the effectiveness of LOHEN in reducing the end-to-end inference latency. As the figure shows, LOHEN reduces the latency by 7–65%, which is equivalent to 1.08–2.88× speedup when applied to FHE-MP-CNN, the state-of-the-art in inference over CKKS. It is also noteworthy that naïvely LCR at all layers, which we refer to as *AllRepack*, may not outperform the state-of-the-art and always have longer latency than LOHEN. AllRepack increases the latency by 38% in the worst case.

Latency Breakdown. Fig. 11 depicts the breakdown of the normalized latency when we use LOHEN (B) and AllRepack (C), when compared to the baseline FHE-MP-CNN (A). For all workloads, LOHEN and AllRepack introduce the latency originating from LCR while reducing the rotation and BTS latency. AllRepack's inefficiency comes from the excessive latency of LCR, which is longer than the rotation and BTS latencies in CR20, CV16, and CSQ. On the contrary, LOHEN-introduced LCR latency is always shorter than the reduced rotation and BTS latencies, resulting in latency improvement.

5.2 Effectiveness on LoLat

LoLat. To better understand the potential effectiveness of LOHEN on corner cases, we evaluate its effectiveness when

Table 15: End-to-end latency of LOHEN applied to LoLat. AllRepack replaces all rotation-based permutations with LCR.

Workload	LoLat	AllRepack	LOHEN
CR20	7.23s	21.40s (2.96×)	5.71s (0.79×)
CV16	22.38s	39.84s (1.78×)	13.43s (0.60×)
CSQ	18.85s	45.05s (2.39×)	14.95s (0.79×)
IR18	85.77s	261.60s (3.05×)	76.34s (0.89×)
ISQ	931.03s	1396.55s (1.50×)	363.10s (0.39×)
IMO	1230.70s	2006.04s (1.63×)	627.66s (0.51×)

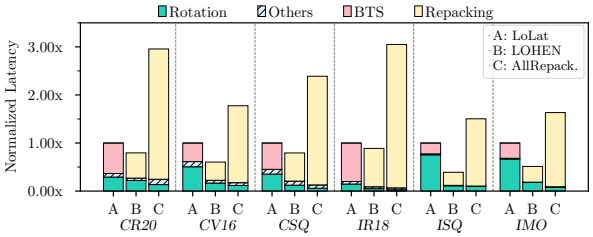
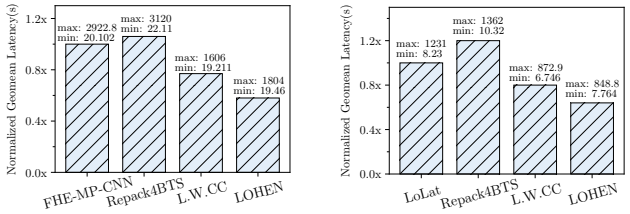


Fig. 12: Latency Breakdown of LOHEN and AllRepack applied to LoLat activation. (Normalized to the baseline).

applied to encrypted neural networks optimized for low inference latency. Specifically, this *LoLat* packs the inputs differently to reduce the number of rotations [30], and uses a simple square operation, x^2 [3, 9], as the activation function to reduce the number of multiplication, resulting in shorter BTS latency. Note that *LoLat* requires re-training the model, resulting in lower accuracy than those using ReLU – Table 14 shows that the use of the square for activation affects the inference accuracy negatively. This is the case where LOHEN is expected to be the least beneficial in improving the latency because LOHEN essentially uses the costs of rotation and BTS as the optimization opportunities.

End-to-end Inference Latency. Table 15 shows the end-to-end inference latency of LOHEN when compared to the unmodified LoLat baseline and the one with AllRepack The reduced rotation latency makes the AllRepack much less effective, increasing the end-to-end latency by three times at most. In contrast, LOHEN automatically adapts to the situation and finds locations to place LCR where it leads for performance enhancements. Overall, LOHEN reduces the latency by 11–61%, corresponding to 1.12–2.56× speedup.

Latency Breakdown. Fig. 12 depicts the latency breakdown of the original LoLat, LOHEN, and AllRepack. The design changes for latency reduction affect all three primary sources of latency, which are rotation, BTS, and ciphertext multiplication. This results in the difference in the relative latency of LoLat's three components when compared to the FHE-MP-CNN. Note that despite the difference, the total rotation latency of LoLat is always lower than that of FHE-MP-CNN. Despite this, LOHEN uses the rotation and BTS latencies as the budget for latency reduction by selectively replacing them with LCR. AllRepack suffers even more for LoLat because the total latency of rotation, which is what AllRepack eliminates, is much shorter than that of FHE-MP-CNN while inducing a similar amount of LCR.



(a) FHE-MP-CNN **(b)** LoLat

Fig. 13: Impact of CC and LCR

5.3 Impact of Design Choices

Fig. 13 shows the impact of individual design choices contributing to LOHEN's performance improvement when running the six workloads we use throughout the evaluation. Due to space limitations, we only report the normalized latencies' geometric mean, with the ranges annotated above each bar. The first bars give the reference point showing FHE-MP-CNN and LoLat results without any LOHEN components. To these, we first apply LCR only as a replacement for BTS, denoted as Repack4BTS. The result suggests that LCR is not an attractive gadget if it is used only to replace BTS, as it increases the latency by 6% and 20%. On top of this, we let LOHEN select the CCs of each set of layers separated by LCR, denoted as L.W.CC. As expected, LCR reveals its value when we take more of its potential by just picking CCs for each layer, resulting in 23% and 20% reduced inference latency, which are equivalent to 1.24× and 1.33× speedup. Capitalizing over another potential permutation while LCR further boosts the performance by reducing the latency by 36% and 42%, which are equivalent to 1.56× and 1.72× speedups. In summary, this study shows that LCR is beneficial only when used for more than a replacement of BTS.

5.4 LOHEN with TFHE layers

Comparison Targets. We evaluate the effectiveness of LOHEN using TFHE-based non-arithmetic layers selectively by comparing it with six alternatives in three categories. In the first category are TFHE-only alternatives, TFHE and SHE [48]. TFHE runs encrypted NN without any approximation, and SHE approximates multiplication with bit-shifts for performance. The second category is the CKKS-only ones, FHE-MP-CNN and LOHEN. The third category is the latest work that uses multi-schemes. HEIR [33] is the unmodified version of the latest work, and HEIR+Opt. is a version that incorporates a mechanism inspired by LCR for lower latency.

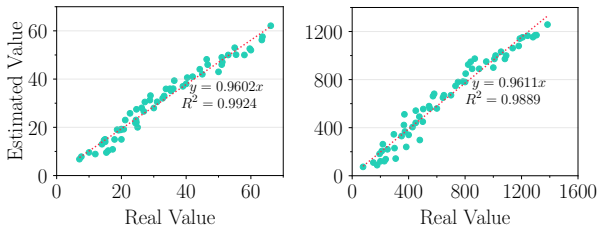
Remark. We note that this optimization is rather straightforward, for which reason we consider this HEIR+Opt. variant as our comparison baseline in favor of HEIR. Also, note that LOHEN can be driven to prioritize either accuracy or latency, as we mentioned earlier.

Results. Table 16 shows accuracy and inference latency when we run LOHEN using TFHE-based non-arithmetic layers selectively when compared to the six possible alternatives.

Table 16: Latency and Accuracy of FHE-MP-NN, LOHEN, and MS-LOHEN when running ISQ and IMO workloads.

	Workload	ISQ (72.4%)		IMO (74.7%)	
		Latency (sec.)	Accuracy (%)	Latency (sec.)	Accuracy(%)
TFHE	TFHE†	0.2M s (96.1×)	72.4% (-0.0%)	0.3M s (97.8×)	74.7% (-0.0%)
	SHE [48]†	0.02M s (8.28×)	69.4% (-3.0%)	0.03M s (10.6×)	70.6% (-4.1%)
CKKS	FHE-MP-NN	2093s (1.00×)	69.1% (-3.3%)	2923s (1.00×)	70.1% (-4.6%)
	LOHEN	1008s (0.48×)	69.1% (-3.3%)	1016s (0.35×)	70.1% (-4.6%)
Multi-Scheme	HEIR [33]	3006s (1.44×)	72.4% (-0.0%)	4481s (1.53×)	74.7% (-0.0%)
	HEIR+Opt.	2271s (1.09×)	72.4% (-0.0%)	3265s (1.12×)	74.7% (-0.0%)
	MS-LOHEN-A	1690s (0.81×)	72.4% (-0.0%)	2056s (0.70×)	74.7% (-0.0%)
	MS-LOHEN-L	1208s (0.58×)	71.2% (-1.2%)	1376s (0.47×)	71.9% (-2.8%)
	MS-LOHEN-B	1297s (0.62×)	72.4% (-0.0%)	1573s (0.54×)	74.7% (-0.0%)

'M' states for Million / †-projection of the first 2 layers / HEIR+Opt. : HEIR with opt. TFHE.
MS-LOHEN-A / L: multi-scheme LOHEN prioritizing accuracy / latency.
MS-LOHEN-B: multi-scheme LOHEN with profile-guided balancing.



(a) CIFAR-10 Inferences **(b)** ImageNet Inferences

Fig. 14: Estimation error analysis of LOHEN.

We only use the two workloads using ImageNet as the data set because the accuracy loss of FHE-MP-CNN for the others is much lower. The results show that MS-LOHEN achieves its goal. MS-LOHEN-A's inference latency is 19% and 30% lower than FHE-MP-CNN, which is lower than all alternatives except for LOHEN and MS-LOHEN-L. It is noteworthy that MS-LOHEN-A achieves this latency without any accuracy loss, outperforming the multi-scheme baseline of HEIR-Opt., with a speedup of 1.34× and 1.59×.

MS-LOHEN-L, on the other hand, trades latency with accuracy, having higher latency and higher accuracy than LOHEN. Nevertheless, MS-LOHEN-L exhibits lower latency than all alternatives, including FHE-MP-CNN, and higher accuracy than LOHEN or FHE-MP-CNN. Unlike these two, four out of the six alternatives (TFHE, SHE, HEIR, and HEIR+Opt.) exhibit higher inference latency (1.09-96.1×) than FHE-MP-CNN or all LOHEN variants. TFHE, HEIR, and HEIR+Opt. do not suffer from accuracy loss, but none of these outperform MS-LOHEN-A, which also exhibits zero accuracy loss. We also observed that our profile-guided approach led MS-LOHEN-B to find a sweet spot between the two other options, even achieving both 1.23–1.85× speedup and higher accuracy compared to when only CKKS is used (FHE-MP-CNN).

5.5 Precision of the Latency Estimation

We evaluate the precision of LOHEN's latency estimation by comparing the estimated cost for various workloads. We repeatedly perform our cost estimation analysis 20 times for each of the 12 NNs from Fig. 11 and Fig. 12 to compare it with their actual latency. As Fig. 14 shows, the estimated latencies are linearly correlated with the actual latency with correlation coefficients of 0.9924 and 0.9889, suggesting that LOHEN's latency estimation provides reliable grounds for selecting FHE operations and configuration for each layer.

6 Discussion

LOHEN for Different Types of NNs. We envision that LOHEN can also be used to improve the performance of other types of NNs such as RNNs or Transformers because they can also be viewed as stacked layers. In most cases, a NN is a sequence of functions or layers each could be arithmetic, non-arithmetic, or both. The computation between weights and features often consists of arithmetic operations over vectors (*e.g.*, convolution) while the activation functions usually consist of non-arithmetic operations (*e.g.*, ReLU and Softmax). For example, RNNs such as LSTM [49] cells also consist of matrix multiplications followed by activation functions and other types of operations over vectors. This sequence of distinct operations is alike the layers in CNNs, where LOHEN can select the layer-wise CC, where to put LCRs and, which layers to replace with TFHE layers.

LOHEN for the Other FHE Schemes. LOHEN can be extended to deal with other FHE schemes such as BGV [34], BFV [50], and FHEW [13] as they share similar traits with the two schemes we use, CKKS and TFHE. BGV and BFV are similar to CKKS in that they use RLWE ciphertexts. They also support packing, rotations, and BTS like CKKS. While the exact latency of individual operations (*e.g.* addition) in such other schemes is not the same as the latency in CKKS, there are some characteristics in common. For example, BTS is often the slowest operation, and rotation remains one of the slow operations with multiplication. Accordingly, LOHEN and its algorithm can be used for NN inferences over BGV or BFV where LCR would perform the same role in CC switching and replacing rotation or BTS operations. LOHEN can also incorporate FHEW in place of TFHE when it uses them to mitigate the accuracy loss from approximated non-arithmetic layers. This is because TFHE roots from FHEW, thus the two schemes have only minor differences in their PBS process. Replacing TFHE with FHEW in LOHEN would only modify the middle phase where the non-arithmetic operations are computed over LWE ciphertexts, which does not affect the overall functionality of LCR.

7 Related Work

Reducing Single-Scheme NNs Inference Latency. FHE-MP-CNN [8], HCNN [3] and multiple prior works [4, 10, 30, 31] designed techniques to reduce the inference latency of single-scheme NNs. Although our evaluation measured how LOHEN further reduces the latency when applied to

FHE-MP-CNN, LOHEN can be applied to any of the aforementioned works and enhance their performance, utilizing the cryptographic repacking to adopt layer-wise CC and avoiding rotations. We also demonstrated that LOHEN successfully helps LoLat, which shares some characteristics with recent studies [10, 30] aimed at lower latency.

Computation-Adaptive Optimization. HECO [2], CHET [5], ngraph-he [7], ELASM [28], HECATE [27], COYOTE [6] introduced automated but limited mechanisms to select a global CC to run a given application. HECO, CHET, and ngraph-he select CC factors with enough levels to compute the whole application without BTS. They are shown to be effective in choosing CCs for relatively shallow models, but the lack of consideration for BTS leaves questions about how they would choose CC for deep NNs, such as ISQ or IMO. On the contrary, LOHEN does not suffer from such restrictions, takes the BTS cost into consideration, and is shown to be effective in reducing the inference latency of deep NNs. Another difference is that LOHEN selects layer-wise CC with the help of cryptographic repacking, further reducing the inference latency. Other works target certain specific parts of the CC selection to reduce the computational overhead with which LOHEN has complementary relations. COYOTE, HECO, and CHET offer automated data layout selection (*i.e.*, packing mechanisms) to enhance the overall performance by avoiding intermediate rotations during CKKS computation. On top of their work, LOHEN can improve performance by avoiding the remaining rotations by repacking. HECATE and ELASM schedule FHE operations to maximize the efficiency for a given ciphertext level limit, while DaCapo [18] schedules BTS to reduce the overhead. LOHEN can exploit their work to estimate the costs of each layer while selecting the optimal CC and operation during our cost analysis.

Multi-Scheme Computation. CHIMERA [32], PEGASUS [14], and HERMES [35] introduced and optimized the scheme conversion gadgets that enable computing on both RLWE and LWE ciphertexts (*i.e.*, CKKS, BFV, and TFHE). Glyph [37] and HEIR [33] use this conversion gadget to perform efficient yet accurate computation by using CKKS for all arithmetic and TFHE for all non-arithmetic layers with a pre-defined CC for each scheme. LCR is built with the building blocks used for the scheme conversion gadgets, primarily to allow adaptive CC switching between RLWE ciphertexts. LOHEN also differs from the latter three in that it takes an analytic approach to adaptively use multiple CCs to enhance the overall performance and obtain a desired accuracy level. LOHEN also provides methods to exploit trade-offs between performance and accuracy, which prior works do not.

Integrating with other techniques. To manage the computational demands of FHE, hybrid approaches combining FHE with MPC have emerged for collaboratively running NNs [51]. This approach involves using FHE for arithmetic layers in the cloud and MPC for non-arithmetic layers at the client end or within a client-controlled Trusted Execution Environment (TEE) [52, 53]. This setting enables the clients to decrypt and encrypt the intermediate results from which they can perform CC switching without cryptographic algorithms [54]. This approach is more desirable if the client is willing to invest in its computing resources and participate in the computation. However, we envision other use cases where the client prefers not to be involved in the computation, where an approach relying only on FHE is the only option, and LOHEN will become a useful building block.

8 Conclusion

We presented LOHEN which improves the performance of NN inference over FHE by enabling to use of layer-wise CCs with LCR. The use of layer-wise CCs is a primary difference of LOHEN from the prior works that use a single, global CC. To enable CC switching, we introduced a new cryptographic gadget called LCR, which is also designed with the ability to replace costly operations such as rotations and bootstrapping. These abilities are found to be essential to make LCR truly improve the encrypted inference performance. We also showed that LOHEN's LCR must be inserted selectively because inserting it to all possible locations degrades the performance. This motivated us to enable LOHEN to automatically identify the locations to insert LCR with the guarantee of performance improvement over global CC. LOHEN is also extended for multi-scheme scenario where the encrypted NN inference can avoid accuracy loss owing to the approximated non-arithmetic layers and enables the developers using LOHEN to drive the selection procedure somewhere in between the performance- and accuracy-oriented LCR and scheme conversion gadget insertion. Our evaluation demonstrated that LOHEN outperforms state-of-the-art works using CKKS by 1.08–2.88×. LOHEN is also shown to achieve 1.34–1.74× speedup when compared to the state-of-the-art multi-scheme works without accuracy loss.

Acknowledgments

This work was partly supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (RS-2023-00277326, RS-2022-00166529), and the Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (No.RS-2024-00439819, AI-Based Automated Vulnerability Detection and Safe Code Generation, No.RS-2024-00437306, Development of Integrated Platform for Expanding and Safely Applying Memory-Safe Languages, No.RS-2024-00438729, Development of Full Lifecycle Privacy-Preserving Techniques using Anonymized Confidential Computing, No.RS-2023-00277060, Development of open edge AI SoC hardware and software platform, 0.1,

No. 2021-0-00528, Development of Hardware-centric Trusted Computing Base and Standard Protocol for Distributed Secure Data Box, 0.1), Samsung Electronics, the BK21 FOUR program of the Education and Research Program for Future ICT Pioneers, Seoul National University in 2024, and the artificial intelligence semiconductor support program to nurture the best talents (IITP-2023-RS-2023-00256081).

References

[1] C. Gentry. Fully homomorphic encryption using ideal lattices. In *Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing*, 2009.

[2] Alexander Viand, Patrick Jattke, Miro Haller, and Anwar Hithnawi. Heco: Automatic code optimizations for efficient fully homomorphic encryption. *arXiv preprint arXiv:2202.01649*, 2022.

[3] Ahmad Al Badawi, Chao Jin, Jie Lin, Chan Fook Mun, Sim Jun Jie, Benjamin Hong Meng Tan, Xiao Nan, Khin Mi Mi Aung, and Vijay Ramaseshan Chandrasekhar. Towards the alexnet moment for homomorphic encryption: Hcnn, the first homomorphic cnn on encrypted data with gpus. *IEEE Transactions on Emerging Topics in Computing*, 9(3):1330–1343, 2020.

[4] Alon Brutzkus, Ran Gilad-Bachrach, and Oren Elisha. Low latency privacy preserving inference. In *International Conference on Machine Learning*, pages 812–821. PMLR, 2019.

[5] Roshan Dathathri, Olli Saarikivi, Hao Chen, Kim Laine, Kristin Lauter, Saeed Maleki, Madanlal Musuvathi, and Todd Mytkowicz. Chet: An optimizing compiler for fully-homomorphic neural-network inferencing. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 142–156, 2019.

[6] Raghav Malik, Kabir Sheth, and Milind Kulkarni. Coyote: A compiler for vectorizing encrypted arithmetic circuits. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pages 118–133, 2023.

[7] Fabian Boemer, Yixing Lao, Rosario Cammarota, and Casimir Wierzynski. ngraph-he: a graph compiler for deep learning on homomorphically encrypted data. In *Proceedings of the 16th ACM international conference on computing frontiers*, pages 3–13, 2019.

[8] Eunsang Lee, Joon-Woo Lee, Junghyun Lee, Young-Sik Kim, Yongjune Kim, Jong-Seon No, and Woosuk Choi. Low-complexity deep convolutional neural networks on fully homomorphic encryption using multiplexed parallel convolutions. In *International Conference on Machine Learning*, pages 12403–12422. PMLR, 2022.

[9] Ran Gilad-Bachrach, Nathan Dowlin, Kim Laine, Kristin Lauter, Michael Naehrig, and John Wernsing. Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy. In *International conference on machine learning*, pages 201–210. PMLR, 2016.

[10] Jaiyoung Park, Michael Jaemin Kim, Wonkyung Jung, and Jung Ho Ahn. Aespa: Accuracy preserving low-degree polynomial activation for fast private inference. *arXiv preprint arXiv:2201.06699*, 2022.

[11] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. A full rns variant of approximate homomorphic encryption. In *Selected Areas in Cryptography–SAC 2018: 25th International Conference, Calgary, AB, Canada, August 15–17, 2018, Revised Selected Papers 25*, pages 347–368. Springer, 2019.

[12] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Tfhe: fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91, 2020.

[13] Léo Ducas and Daniele Micciancio. Fhew: bootstrapping homomorphic encryption in less than a second. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 617–640. Springer, 2015.

[14] Wen-jie Lu, Zhicong Huang, Cheng Hong, Yiping Ma, and Hunter Qu. Pegasus: bridging polynomial and non-polynomial evaluations in homomorphic encryption. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 1057–1073. IEEE, 2021.

[15] CrytptoLab. Official HEaaN Library, 2022. https://heaan.it/.

[16] Microsoft seal (release 3.7). https://github.com/Microsoft/SEAL, 2021.

[17] Youngjin Bae, Jung Hee Cheon, Wonhee Cho, Jaehyung Kim, and Taekyung Kim. Meta-bts: Bootstrapping precision beyond the limit. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 223–234, 2022.

[18] Seonyoung Cheon, Yongwoo Lee, Dongkwan Kim, Ju Min Lee, Sunchul Jung, Taekyung Kim, Dongyoon Lee, and Hanjun Kim. Dacapo: Automatic bootstrapping management for efficient fully homomorphic encryption.

[19] Zama. Concrete ML: a privacy-preserving machine learning library using fully homomorphic encryption for data scientists, 2022. https://github.com/zama-ai/concrete-ml.

[20] Zama. TFHE-rs: A Pure Rust Implementation of the TFHE Scheme for Boolean and Integer Arithmetics Over Encrypted Data, 2022. https://github.com/zama-ai/tfhe-rs.

[21] I. Chillotti et al. Programmable bootstrapping enables efficient homomorphic inference of deep neural networks. In *Cyber Security Cryptography and Machine Learning: 5th International Symposium*. Springer, 2021.

[22] A. Sanyal et al. Tapas: Tricks to accelerate (encrypted) prediction as a service. In *International conference on machine learning*. PMLR, 2018.

[23] Jean-Philippe Bossuat, Rosario Cammarota, Jung Hee Cheon, Ilaria Chillotti, Benjamin R Curtis, Wei Dai, Huijing Gong, Erin Hales, Duhyeong Kim, Bryan Kumara, et al. Security guidelines for implementing homomorphic encryption. *Cryptology ePrint Archive*, 2024.

[24] Joon-Woo Lee, HyungChul Kang, Yongwoo Lee, Woosuk Choi, Jieun Eom, Maxim Deryabin, Eunsang Lee, Junghyun Lee, Donghoon Yoo, Young-Sik Kim, et al. Privacy-preserving machine learning with fully homomorphic encryption for deep neural network. *IEEE Access*, 10:30039–30054, 2022.

[25] Jung Hee Cheon, Minsik Kang, Taeseong Kim, Junyoung Jung, and Yongdong Yeo. High-throughput deep convolutional neural networks on fully homomorphic encryption using channel-by-channel packing. *Cryptology ePrint Archive*, 2023.

[26] Miran Kim, Xiaoqian Jiang, Kristin Lauter, Elkhan Ismayilzada, and Shayan Shams. Secure human action recognition by encrypted neural network inference. volume 13, page 4799. Nature Publishing Group UK London, 2022.

[27] Yongwoo Lee, Seonyeong Heo, Seonyoung Cheon, Shinnung Jeong, Changsu Kim, Eunkyung Kim, Dongyoon Lee, and Hanjun Kim. Hecate: Performance-aware scale optimization for homomorphic encryption compiler. In *2022 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 193–204. IEEE, 2022.

[28] Yongwoo Lee, Seonyoung Cheon, Dongkwan Kim, Dongyoon Lee, and Hanjun Kim. {ELASM}:{Error-Latency-Aware} scale management for fully homomorphic encryption. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 4697–4714, 2023.

[29] Roshan Dathathri, Blagovesta Kostova, Olli Saarikivi, Wei Dai, Kim Laine, and Madan Musuvathi. Eva: An encrypted vector arithmetic language and compiler for efficient homomorphic computation. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 546–561, 2020.

[30] Donghwan Kim, Jaiyoung Park, Jongmin Kim, Sangpyo Kim, and Jung Ho Ahn. Hyphen: A hybrid packing method and optimizations for homomorphic encryption-based neural networks. *arXiv preprint arXiv:2302.02407*, 2023.

[31] Jung Hee Cheon, Minsik Kang, Taeseong Kim, Junyoung Jung, and Yongdong Yeo. High-throughput deep convolutional neural networks on fully homomorphic encryption using channel-by-channel packing. *Cryptology ePrint Archive*, 2023.

[32] Christina Boura, Nicolas Gama, Mariya Georgieva, and Dimitar Jetchev. Chimera: Combining ring-lwe-based fully homomorphic encryption schemes. *Journal of Mathematical Cryptology*, 14(1):316–338, 2020.

[33] Song Bian, Zian Zhao, Zhou Zhang, Ran Mao, Kohei Suenaga, Yier Jin, Zhenyu Guan, and Jianwei Liu. Heir: A unified representation for cross-scheme compilation of fully homomorphic computation. *Cryptology ePrint Archive*, 2023.

[34] Craig Gentry, Shai Halevi, Chris Peikert, and Nigel P Smart. Field switching in bgv-style homomorphic encryption. *Journal of Computer Security*, 21(5):663–684, 2013.

[35] Youngjin Bae, Jung Hee Cheon, Jaehyung Kim, Jai Hyun Park, and Damien Stehlé. Hermes: Efficient ring packing using mlwe ciphertexts and application to transciphering. In *Annual International Cryptology Conference*, pages 37–69. Springer, 2023.

[36] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. Bootstrapping for approximate homomorphic encryption. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 360–384. Springer, 2018.

[37] Qian Lou, Bo Feng, Geoffrey Charles Fox, and Lei Jiang. Glyph: Fast and accurately training deep neural networks on encrypted data. *Advances in neural information processing systems*, 33:9193–9202, 2020.

[38] Guanpeng Li, Siva Kumar Sastry Hari, Michael Sullivan, Timothy Tsai, Karthik Pattabiraman, Joel Emer, and Stephen W Keckler. Understanding error propagation in deep learning neural network (dnn) accelerators and applications. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–12, 2017.

[39] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[40] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.

[41] Tensorsandbox, 2017. https://github.com/dacorvo/tensorsandbox.

[42] Forrest N Iandola, Song Han, Matthew W Moskewicz, Khalid Ashraf, William J Dally, and Kurt Keutzer. Squeezenet: Alexnet-level accuracy with 50x fewer parameters and< 0.5 mb model size. *arXiv preprint arXiv:1602.07360*, 2016.

[43] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4510–4520, 2018.

[44] DESILO. Liberate.FHE: A New FHE Library for Bridging the Gap between Theory and Practice with a Focus on Performance and Accuracy, 2023. https://github.com/Desilo/liberate-fhe.

[45] SNU CNL. Fhe-mp-cnn, 2021. https://github.com/snu-ccl/FHE-MP-CNN.

[46] A gpu implementation of fully homomorphic encryption on torus. https://github.com/nucypher/nufhe.

[47] François Chollet. Keras. https://keras.io, 2015.

[48] Qian Lou and Lei Jiang. She: A fast and accurate deep neural network for encrypted data. *Advances in Neural Information Processing Systems*, 32, 2019.

[49] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

[50] Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Paper 2012/144, 2012. https://eprint.iacr.org/2012/144.

[51] Chiraag Juvekar, Vinod Vaikuntanathan, and Anantha Chandrakasan. {GAZELLE}: A low latency framework for secure neural network inference. In *27th USENIX Security Symposium (USENIX Security 18)*, pages 1651–1669, 2018.

[52] Ankur Dave, Chester Leung, Raluca Ada Popa, Joseph E Gonzalez, and Ion Stoica. Oblivious coopetitive analytics using hardware enclaves. In *Proceedings of the Fifteenth European Conference on Computer Systems*, pages 1–17, 2020.

[53] Olga Ohrimenko, Felix Schuster, Cédric Fournet, Aastha Mehta, Sebastian Nowozin, Kapil Vaswani, and Manuel Costa. Oblivious multi-party machine learning on trusted processors. In *USENIX Security Symposium*, volume 16, pages 10–12, 2016.

[54] Qian Lou, Song Bian, and Lei Jiang. Autoprivacy: Automated layer-wise parameter selection for secure neural network inference. *Advances in Neural Information Processing Systems*, 33:8638–8647, 2020.

Algorithm 2: Ringswitch (to a lower degree) [35]

Input: $ct_{in} \in RLWE_{q,N}$, $\mathbf{swk}_{s \to s'}$
Output: $(ct^i_{out})_{0 \le i < \frac{N}{n}} \in RLWE_{q,n}$

$ct' \longleftarrow \mathsf{KS}(ct_{in}, \mathbf{swk}_{s \to s'})$
$(ct^i_{out})_{0 \le i < \frac{N}{n}} \longleftarrow \mathsf{Split}(ct')$
return $(ct^i_{out})_{0 \le i < \frac{N}{n}}$

Algorithm 3: LWE-Extract [14]

Input: $ct_{in} \in RLWE_{q,n}$
Output: $(ct_i)_{0 \le i < n} \in LWE^n_q$

for $i = 0$ **to** $n-1$ **do**
  $ct_i \longleftarrow \mathsf{Extract}^i(ct_{in})$
return $(ct_i)_{0 \le i < n}$

Algorithm 4: RLWE-Pack [35]

Input: $(ct^i_{in})_{0 \le i < n} \in LWE^n_q$, $\left(\mathbf{swk}^i_{s' \to s}\right)_{0 \le i < n}$
Output: $ct_{out} \in RLWE_{q,n}$

Let $ct^i_{in} = (b_i, a^0_i, a^1_i, \cdots, a^{n-1}_i) \in \mathbb{Z}_q \times \mathbb{Z}^n_q$
$\alpha_j(X) \leftarrow \sum_{i=0}^{n-1} a^j_i X^i$ for all $j < n$
$ct_{out} \leftarrow \mathsf{modDown}(\sum_{j=0}^{n-1} \mathsf{modUp}(\alpha_j(X)) \cdot \mathbf{swk}^j_{s' \to s}) + (0, \sum_{i=0}^{n-1} b_i X^{n-1})$
return ct_{out}

Algorithm 5: LCR

Input: $ct_{in} \in RLWE_{q,N}$
Output: $ct_{out} \in RLWE_{Q,N}$

1 $ct' \in RLWE_{q',N} \longleftarrow \mathsf{StC}(RLWE_{q,N})$;
2 $\{ct''_0, ct''_1, \cdots, ct''_{\frac{N}{n}-1}\} \in RLWE_{q',n} \longleftarrow \mathsf{Split}(\mathsf{KS}(ct', \mathbf{swk}_{s \to s'}))$;
3 **for** $i = 0$ **to** $\frac{N}{n} - 1$ **do**
4 **for** $j = 0$ **to** $n-1$ **do**
5 $ct'''_{i \cdot n + j} \in LWE^n_{q'} \longleftarrow \mathsf{Extract}^j(ct''_i)$;
6 **Permute**(ct''');
7 **for** $i = 0$ **to** $\frac{N}{n} - 1$ **do**
8 $ct''''_i \in RLWE_{q',n} \longleftarrow$ RLWE-Pack$((ct'''_j)_{i \cdot n \le j < (i+1) \cdot n})$;
9 $\tilde{ct} \in RLWE_{q',N} \longleftarrow \mathsf{KS}(\mathsf{Combine}((ct''''_i)_{0 \le i < \frac{N}{n}}), \mathbf{swk}_{s' \to s})$;
10 $ct_{out} \in RLWE_{Q,N} \longleftarrow \mathsf{EvalMod}(\mathsf{CtS}(\mathsf{ModRaise}(\tilde{ct})))$;
11 **return** ct_{out}

A CC Switching Routines & LCR

Algo. 2 shows an algorithm of Ringswitch RLWE ciphertext in degree N to smaller n. Ringswitch from a large ring to smaller rings consists of key-switching to a small-degree key s' and Split. In the opposite direction, from a smaller rings to a larger ring, the process includes Combine, followed by key-switching to a large-degree key s. Both the functions Split and Combine rearrange the coefficients to produce ciphertexts.

LWE-Extract brings out i-th coefficient of RLWE ciphertext, which contains the i-th message as shown in Algo. 3. The function $\mathsf{Extract}^i$ takes RLWE ciphertext (b,a) as an input and extracts i-th LWE ciphertext $(b_i, a^{(i)})$ where $a^{(i)} = (a_i, a_{i-1}, ..., a_0, -a_{i-1}, ..., -a_{i+1})$. Note that this process simply rearranged the values, incurring minimal additional cost.

Algo. 4 depicts RLWE-Pack that comprises of three sub-operations: modDown, modUp and hadamard multiplication, which are used in the key switching operation. modUp turns a polynomial in $R_{q,N}$ to R_N and applies modulo pq, modDown is an approximate division by p which maps polynomial a into $(a - [a]_p)/p$. RLWE-Pack first regards given n LWE ciphertexts as a matrix $M \in \mathbb{Z}^{n \times n}_q$ where each rows are the LWE ciphertexts. It defines j-th column of M as $\alpha_j(X)$ and then sums the results of modUp of $\alpha_j(X)$ and applies hadamard multiplication with switching key and modDown, ends up with adding b. Consequently, we obtain the RLWE ciphertext that holds the message of LWE ciphertexts in the given order.

The full algorithm of LCR is outlined in Algo. 5. It begins with StC to convert slot-encoded ciphertext into a coefficient-encoded format (line 1). Then, it decreases the ring degree and generate N/n RLWE ciphertexts in the lower ring degree by switching key and splitting coefficients (line 2). Then the LWE ciphertexts are extracted from each N/n RLWE ciphertexts (line 3-line 5). During the Permute (line 6), LOHEN performs not only permutation given LWE ciphertexts but also evaluates TFHE-based non-arithmetic operations. After Applying RLWE-Pack on the re-ordered N/n groups of n LWEs, which generates n RLWEs in ring degree n, we combine these n RLWEs and switch the ring back to N (line 7-line 9). Finally, We employ three sequential routines to recover the modulus, which follows the same process as in BTS (line 10).